

DD: BIL-01-CN-01 — Credit & Debit Note Application

Developer implementation guide: DD_BIL-01-CN-01-Note_Application-DEV_IMPL_GUIDE-v1.0.md.

1. Purpose

This rewritten DD is the developer-facing version of the module contract
DD_BIL-01-CN-01-Note_Application-v1.0.md.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing | **Version:** v1.0 Satellite of **BIL-01 Charging Engine (Core)**. Owns the application of credit notes and debit notes issued by BIL-02-ADJ-01 (via GEN-01). A credit note reduces what the account owes — applied to an outstanding POSTPAID invoice or credited to a PREPAID wallet. A debit note increases what the account owes — applied to an open or paid POSTPAID invoice or debited from a PREPAID wallet. CN-01 consumes CreditNoteIssued / DebitNoteIssued events from GEN-01, applies them, writes to its ledger, and emits CreditNoteApplied / DebitNoteApplied events that BIL-02-STA-01 consumes for status transitions. **Parent DD:** BIL-01-Charging_Engine-Core. Shares the billing-charging module deployment, the settlement adapters (to BIL-05 for wallet ops), and reads from BIL-02 (invoice.outstanding_amount). CN-01 owns its own ledger table (note_application_ledger) and Kafka consumer subscriptions. No new REST surface — CN-01 is purely event-driven. **Sibling DDs:** BIL-01-PAY-01 (payment application; PAY-01 owns account_credit_balance which CN-01 may write to for credit note surplus). **Version history:** see CHANGELOG_BIL-01-CN-01-Note_Application.md in the same directory.

The goal of this rewrite is to make the DD easier for a mid-level developer to implement without losing the detailed source contract.

2. Implementation Scope

This DD should be implemented as part of the owning SOPHIX capability, not as an isolated service unless the deployment architecture explicitly separates that capability.

Implement this DD with these rules:

- Use the owning module APIs/repositories for authoritative writes.
- Use approved internal APIs/client wrappers when reading another module's data.
- Do not query another module's database tables directly from business flow code.
- Treat worker names as logical Camunda external-task topics, not separate deployments.
- One worker application may handle multiple topics, but metrics, retries, and logs must remain visible per topic.
- Emit success events only after the authoritative state commit succeeds.
- Use outbox publishing for Kafka events.

3. Runtime Metadata

Item	Value
Source file	/Users/macbook/Downloads/Sophix_V3_BSS_DDs_MD_2026-05-27/05_billing/DD_BIL-01-CN-01-Note_Application-v1.0.md
Version	v1.0
DD type	module contract
BPMN/process keys	Not explicitly defined
Drools packages	Not explicitly defined

4. Runtime Contracts To Implement

4.1 APIs

- POST /api/wallets/credit
- POST /api/wallets/debit

4.2 Tables

- No CREATE TABLE block is explicitly declared in this DD.

For each table in this DD, implement the schema with:

- a short table comment explaining what the table is for;
- column comments or migration notes explaining each field;
- constraints and indexes from the DD;
- seed data where the DD provides operator-specific sample rows.

4.3 Worker Topics

- billing-charging

Worker-topic guidance:

- A BPMN service task uses the topic.
- A worker application subscribes to the topic.
- The topic handler validates input variables, calls the owning module/API, writes outputs back to Camunda, and records metrics.
- Do not treat the topic string as a shell command or Kubernetes deployment name.

4.4 Events

- No domain events are explicitly declared in this DD.

Event guidance:

- Write events through the transactional outbox.
- Include operation id, aggregate id, operator code, actor, and correlation data.
- Consumers should be able to rebuild downstream state without querying workflow internals.

5. Developer Implementation Checklist

1. Add or verify database migrations and seed rows.
2. Add or verify config rows for the operator/module.
3. Implement request/command DTOs and validation.
4. Implement idempotency and in-flight operation checks where the DD requires them.
5. Implement Drools fact mapping and package deployment where rules are used.
6. Implement BPMN process, service tasks, gateways, timers, and message catches where the DD is a flow.
7. Implement worker-topic handlers using owning module APIs.
8. Implement compensation paths and manual recovery paths.
9. Emit success/rejected events through the outbox.
10. Add smoke tests for happy path, validation failure, dependency failure, and retry/idempotency.

6. Infrastructure And AWS Notes

Deploy this DD with the existing SOPHIX platform components unless the owning module requires isolation:

Concern	Recommendation
API/runtime	Run inside the owning module deployment or existing workflow API pods.
Workers	Consolidate low-volume topics into shared worker apps; keep per-topic metrics.
Database	Use the owning module PostgreSQL schema and migration pipeline.
Kafka	Use the foundation Kafka/MSK setup and transactional outbox.
Camunda	Deploy BPMN files through the workflow deployment pipeline.
Drools	Deploy KJAR/rule artifacts through the approved rules pipeline.

Concern	Recommendation
Secrets	Use the platform secret manager; on AWS prefer Secrets Manager/Parameter Store with IRSA.
Observability	Alert on stuck operations, failed workers, outbox backlog, and external dependency failures.

7. Original DD Detail Preserved

The following section preserves the detailed source contract for implementation and review.

Verdict: ■ New | **Wave:** 1 | **Group II:** Billing | **Version:** v1.0

Satellite of **BIL-01 Charging Engine (Core)**. Owns the application of credit notes and debit notes issued by BIL-02-ADJ-01 (via GEN-01). A credit note reduces what the customer owes — applied to an outstanding POSTPAID invoice or credited to a PREPAID wallet. A debit note increases what the customer owes — applied to an open or paid POSTPAID invoice or debited from a PREPAID wallet. CN-01 consumes `CreditNoteIssued` / `DebitNoteIssued` events from GEN-01, applies them, writes to its ledger, and emits `CreditNoteApplied` / `DebitNoteApplied` events that BIL-02-STA-01 consumes for status transitions.

Parent DD: BIL-01-Charging_Engine-Core. Shares the billing-charging module deployment, the settlement adapters (to BIL-05 for wallet ops), and reads from BIL-02 (`invoice.outstanding_amount`). CN-01 owns its own ledger table (`note_application_ledger`) and Kafka consumer subscriptions. No new REST surface — CN-01 is purely event-driven.

Sibling DDs: BIL-01-PAY-01 (payment application; PAY-01 owns `account_credit_balance` which CN-01 may write to for credit note surplus).

Version history: see `CHANGELOG_BIL-01-CN-01-Note_Application.md` in the same directory.

Glossary

Term	Meaning
Credit note	A document issued by BIL-02-ADJ-01 / GEN-01 that reduces what the customer owes. Source events: dispute resolved in customer's favor, billing error correction, SLA compensation, goodwill credit.
Debit note	A document issued by BIL-02-ADJ-01 / GEN-01 that increases what the customer owes. Source events: under-billing correction, late fees added to a paid invoice, recovery from a previously-issued over-credit.
Note application	The act of applying a credit or debit note to a customer's balance. For POSTPAID: reduces or increases <code>invoice.outstanding_amount</code> (the actual status change is owned by STA-01). For PREPAID: credits or debits the wallet via BIL-05.
Target invoice	When a note is scoped to a specific invoice (typical for line-level corrections, billing errors, late-fee additions). The note event payload from GEN-01 includes <code>target_invoice_id</code> in this case.
Auto-allocation	When a credit note has no specific target invoice (e.g., a goodwill credit not tied to one specific invoice), CN-01 applies it across the customer's open invoices using the same allocation policy as PAY-01.
Surplus	When a credit note's amount exceeds the target invoice's outstanding (or the account's total outstanding for auto-allocated credits), the remaining amount goes to <code>account_credit_balance</code> per the operator's overpayment policy (reused from PAY-01's <code>pay_01_config</code>).
Failed application	A debit note against a PREPAID wallet that doesn't have sufficient balance. CN-01 cannot make a wallet negative; the application is logged with <code>status = FAILED</code> , an event is emitted, and admin must reconcile manually.

A — Target

1. What this process is about

When BIL-02-ADJ-01 approves an adjustment, BIL-02-GEN-01 issues the corresponding credit note or debit note (this is BIL-02 family business). At the moment of issuance, GEN-01 emits a Kafka event (`CreditNoteIssued` or `DebitNoteIssued`) carrying the note's identity, target, amount, and reason. CN-01 consumes that event and **applies** the

note — meaning it actually moves the money:

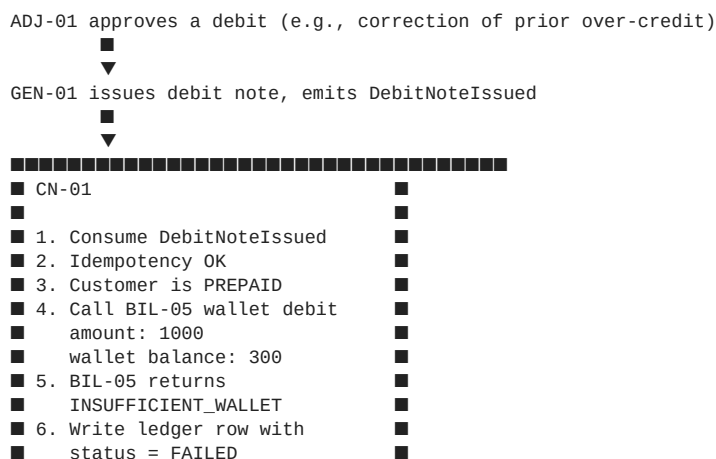
- **POSTPAID credit note:** reduces `invoice.outstanding_amount`. If the credit exceeds outstanding, surplus goes to `account_credit_balance` (managed by PAY-01).
- **POSTPAID debit note:** increases `invoice.outstanding_amount`. May take a previously-PAID invoice back to PENDING/OVERDUE.
- **PREPAID credit note:** calls `POST /api/wallets/credit` on BIL-05. Customer's wallet balance grows.
- **PREPAID debit note:** calls `POST /api/wallets/debit` on BIL-05. If insufficient balance, the application fails and admin must reconcile.

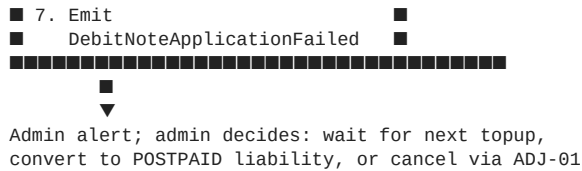
The application is recorded in `note_application_ledger`. An event (`CreditNoteApplied` / `DebitNoteApplied`) is then emitted, which STA-01 consumes to transition invoice status (credit note clearing outstanding → PAID; debit note adding outstanding → PENDING/OVERDUE depending on due date).

Flow overview for a POSTPAID credit note



Flow overview for a PREPAID debit note that fails





What CN-01 explicitly does not own

- **Adjustment workflow** (proposal → approval) — owned by BIL-02-ADJ-01
- **Note generation / issuance** — owned by BIL-02-GEN-01. The credit note / debit note document and its note_id are issued by GEN-01 BEFORE CN-01 sees the event.
- **Charging** (outbound) — owned by BIL-01 Core
- **Payment receipt** — owned by BIL-01-PAY-01
- **Invoice status transitions** — owned by BIL-02-STA-01
- **Wallet balance / top-up** — owned by BIL-05
- **account_credit_balance table** — owned by BIL-01-PAY-01. CN-01 calls into PAY-01's CreditBalanceService when credit note surplus needs to be held as account balance.
- **Refund to gateway** — out of scope for CN-01. If an operator wants to actually return money to the customer's payment method, that's a separate refund flow handled by the gateway adapter, not CN-01's application logic.

2. Business rules

Rules grouped: CN (note consumption), AP (application), FA (failure handling), EV (event emission).

Rule group CN — Note consumption

ID	Rule	Triggered on
R-CN-01-CN-1	CN-01 consumes two Kafka events from GEN-01: CreditNoteIssued and DebitNoteIssued. Topic: sophix.billing.invoice.notes (or as configured per FOUNDATION_KAFKA.md).	Per note event
R-CN-01-CN-2	The note event payload (defined by GEN-01) includes: note_id, note_type (CREDIT / DEBIT), customer_id, operator_code, target_invoice_id (optional — when the note is scoped to a specific invoice; required for debit notes per ADJ-01 rules), target_wallet_ref (optional — used by ADJ-01 to direct PREPAID credit notes to a specific wallet when the customer has multiple), amount (always positive), currency, adjustment_request_id (FK to ADJ-01), reason_code, issued_at.	Per note event
R-CN-01-CN-3	Idempotency on note_id. If a CreditNoteIssued / DebitNoteIssued event with the same note_id has already been applied (status IN ('APPLIED', 'FAILED')), CN-01 acks the event and returns without re-applying. GEN-01 may republish events on recovery.	Per consume
R-CN-01-CN-4	Customer billing mode is determined at application time. CN-01 reads from the customer's primary subscription's billingMode. If the note has a target_invoice_id, the invoice's billing mode is the source of truth (always POSTPAID for an invoice). If the note has a target_wallet_ref, it's PREPAID by definition.	Per application

Rule group AP — Application

ID	Rule	Triggered on
R-CN-01-AP-1	POSTPAID — credit note application. If <code>target_invoice_id</code> is provided: reduce that invoice's <code>outstanding_amount</code> by <code>min(amount, outstanding_amount)</code> . If <code>amount > outstanding</code> , surplus is forwarded to PAY-01's <code>CreditBalanceService</code> to credit <code>account_credit_balance</code> (per operator's <code>overpayment_policy</code> — same policy reused for credit note surplus). If <code>target_invoice_id</code> is NOT provided (auto-allocated credit note): apply across the account's open invoices using PAY-01's allocation policy (typically FIFO by due date).	POSTPAID credit note
R-CN-01-AP-2	POSTPAID — debit note application. Debit notes are always tied to a <code>target_invoice_id</code> (ADJ-01 requires the source invoice in the proposal). CN-01 increases that invoice's <code>outstanding_amount</code> by <code>amount</code> . There is no "surplus" concept for debit notes — the amount is whatever ADJ-01 approved. Status implications (PAID invoice with a debit note becoming <code>PARTIALLY_PAID</code> again; <code>PENDING</code> staying <code>PENDING</code> with higher outstanding; <code>OVERDUE</code> staying <code>OVERDUE</code>) are owned by STA-01 via the <code>DebitNoteApplied</code> event.	POSTPAID debit note
R-CN-01-AP-3	PREPAID — credit note application. CN-01 calls <code>POST /api/wallets/credit</code> on BIL-05 with <code>{customer_id, wallet_ref, amount, currency, note_id, idempotency_key}</code> . The <code>wallet_ref</code> defaults to the customer's primary wallet unless ADJ-01 specified a <code>target_wallet_ref</code> (forwarded in the note event). BIL-05 credits the wallet and returns the new balance.	PREPAID credit note
R-CN-01-AP-4	PREPAID — debit note application. CN-01 calls <code>POST /api/wallets/debit</code> on BIL-05. If BIL-05 returns <code>INSUFFICIENT_WALLET</code> , the application status becomes <code>FAILED</code> (rule group FA). The wallet is never made negative.	PREPAID debit note
R-CN-01-AP-5	Each application writes ONE <code>note_application_ledger</code> row capturing: <code>note_id</code> , <code>note_type</code> , <code>target_kind</code> (<code>INVOICE / WALLET / CREDIT_BALANCE</code>), <code>target_id</code> (<code>invoice_id / wallet_ref / NULL</code> for <code>CREDIT_BALANCE</code>), <code>applied_amount</code> , <code>target_balance_before</code> , <code>target_balance_after</code> , <code>status</code> . For an auto-allocated credit note that spans multiple invoices, ONE row per invoice is written (matching PAY-01's allocation pattern). For a credit note that produces surplus, TWO rows are written: one for the invoice application, one for the <code>credit_balance</code> application. Atomicity: all rows for one note commit in one DB transaction.	Every application
R-CN-01-AP-6	When CN-01 applies surplus to <code>account_credit_balance</code> , it does NOT write directly to PAY-01's table. CN-01 calls <code>CreditBalanceService.credit(account_id, customer_id, amount, reason: CREDIT_NOTE_SURPLUS, note_id)</code> — PAY-01's service writes the update to <code>account_credit_balance</code> and emits <code>CreditBalanceAdjusted</code> event. This preserves the table-ownership boundary.	Per credit note surplus

Rule group FA — Failure handling

ID	Rule	Triggered on
R-CN-01-FA-1	PREPAID debit note on insufficient wallet: CN-01 writes a <code>note_application_ledger</code> row with <code>status = FAILED</code> , <code>failure_reason = WALLET_INSUFFICIENT_BALANCE</code> , the attempted amount, and the wallet's current balance. Emits <code>DebitNoteApplicationFailed</code> event. The note is NOT auto-retried — admin must decide whether to wait for the next top-up, convert the debit to a POSTPAID liability (manual ADJ-01 workflow), or cancel the original adjustment.	PREPAID debit on insufficient wallet
R-CN-01-FA-2	POSTPAID note on a missing invoice: if <code>target_invoice_id</code> is provided but the invoice doesn't exist (was deleted? wrong ID in the proposal?), CN-01 writes a <code>note_application_ledger</code> row with <code>status = FAILED</code> , <code>failure_reason = INVOICE_NOT_FOUND</code> . Emits <code>NoteApplicationFailed</code> event. Admin must investigate.	Invalid <code>target_invoice_id</code>
R-CN-01-FA-3	POSTPAID note on a CANCELLED or WRITTEN_OFF invoice: if the target invoice's status is <code>CANCELLED</code> or <code>WRITTEN_OFF</code> , the application is rejected with <code>status = FAILED</code> , <code>failure_reason = INVOICE_NOT_APPLIABLE</code> . Credit notes on cancelled invoices don't make sense (nothing to credit); debit notes on written-off invoices don't make sense (the operator has already given up on recovery). Admin handles via ADJ-01 re-evaluation.	Invalid invoice status
R-CN-01-FA-4	Auto-allocated credit note on account with no open invoices: if <code>target_invoice_id</code> is null and the account has no open invoices, the full credit goes to <code>account_credit_balance</code> directly (no invoice rows). Status <code>APPLIED</code> .	Auto credit, no open invoices
R-CN-01-FA-5	Currency mismatch: if the note's currency doesn't match the target (invoice currency / wallet currency / customer currency for credit balance), the application is rejected with <code>status = FAILED</code> , <code>failure_reason = CURRENCY_MISMATCH</code> . Multi-currency is out of scope; this should not happen if ADJ-01 generated the note correctly.	Defensive

Rule group EV — Event emission

ID	Rule	Triggered on
R-CN-01-EV-1	CN-01 emits the following events (envelope: <code>eventType</code> , <code>aggregateId</code> , <code>timestamp</code> , <code>payload</code>). Topics: <code>sophix.billing.creditnote.applied</code> , <code>sophix.billing.debitnote.applied</code> , <code>sophix.billing.debitnote.application-failed</code> , <code>sophix.billing.note.application-failed</code> (generic for non-debit-specific failures).	Various
R-CN-01-EV-2	<code>CreditNoteApplied</code> payload: <code>note_id</code> , <code>adjustment_request_id</code> , <code>customer_id</code> , <code>operator_code</code> , <code>target</code> (one of { <code>type: INVOICE</code> , <code>invoice_id</code> , <code>applied_amount</code> , <code>outstanding_after</code> }; { <code>type: WALLET</code> , <code>wallet_ref</code> , <code>applied_amount</code> , <code>balance_after</code> }; { <code>type: CREDIT_BALANCE</code> , <code>applied_amount</code> , <code>balance_after</code> }), <code>currency</code> , <code>applied_at</code> . Emitted PER application row — a credit note that splits across invoice + credit balance produces TWO events. STA-01 consumes <code>INVOICE-target</code> events.	Per credit note application
R-CN-01-EV-3	<code>DebitNoteApplied</code> payload: same shape as <code>CreditNoteApplied</code> but <code>eventType = DebitNoteApplied</code> . STA-01 consumes to handle status implications.	Per debit note application

ID	Rule	Triggered on
R-CN-01-EV-4	DebitNoteApplicationFailed payload: note_id, adjustment_request_id, customer_id, operator_code, target (WALLET only, since this is the PREPAID insufficient-balance case), attempted_amount, wallet_balance, currency, reason = WALLET_INSUFFICIENT_BALANCE, attempted_at.	Per failed PREPAID debit
R-CN-01-EV-5	NoteApplicationFailed payload: note_id, note_type, customer_id, operator_code, failure_reason (INVOICE_NOT_FOUND / INVOICE_NOT_APPLIABLE / CURRENCY_MISMATCH), attempted_at. Generic failure event for non-wallet failures.	Per non-wallet failure

3. Module owner and impacted modules

Role	Module	What it does
Owner	billing-charging module (shared with BIL-01 Core and BIL-01-PAY-01), sub-package note	Owns note_application_ledger table. Kafka consumer for CreditNoteIssued / DebitNoteIssued. Calls BIL-05 for wallet operations and PAY-01's CreditBalanceService for credit note surplus. No REST surface (purely event-driven).
Impacted (callees)	BIL-02-Core (read invoice.outstanding_amount), BIL-05 (wallet credit / debit), BIL-01-PAY-01 (CreditBalanceService for surplus)	Receive synchronous calls.
Impacted (callers)	BIL-02-GEN-01 (publishes CreditNoteIssued / DebitNoteIssued events)	Drive CN-01's flow.
Impacted (downstream consumers)	BIL-02-STA-01 (consumes CreditNoteApplied / DebitNoteApplied to transition invoice status), BIL-04 Collections (consumes failure events for visibility), DD_NOT-01 (consumes for customer notifications), CVM, analytics, finance ETL	Consume CN-01 emission events.
Engines	Kafka (consumer + producer). No Drools, no Camunda.	—

4. Foundation references

DB → FOUNDATION_DB.md · Kafka → FOUNDATION_KAFKA.md · Auth → FOUNDATION_AUTH.md.

Technical design

Code structure

```

modules/billing-charging/
  src/main/java/com/sophix/billing/charging/note/
    service/
      NoteApplicationService.java      ← entry-point handler: dedup, route by billing mode
      PostpaidNoteApplicator.java      ← invoice outstanding update + surplus to credit balance
      PrepaidNoteApplicator.java       ← wallet credit / debit via BIL-05
      NoteApplicationLedgerWriter.java
    kafka/
      NoteEventConsumer.java           ← consumes CreditNoteIssued / DebitNoteIssued
    domain/
      NoteApplication.java
      NoteType.java                   ← enum: CREDIT, DEBIT
      ApplicationStatus.java           ← enum: APPLIED, FAILED
      TargetKind.java                  ← enum: INVOICE, WALLET, CREDIT_BALANCE
      FailureReason.java               ← enum: WALLET_INSUFFICIENT_BALANCE, INVOICE_NOT_FOUND, etc.
    repository/
      NoteApplicationLedgerRepository.java
    event/
      CreditNoteAppliedEventPublisher.java
      DebitNoteAppliedEventPublisher.java
      DebitNoteApplicationFailedEventPublisher.java
      NoteApplicationFailedEventPublisher.java
  src/main/resources/

```

Data model

note_application_ledger

Append-only ledger of every credit / debit note application.

Column	Type	Notes
id	TEXT PK	ULID. Example: na_01HZX9K8M7Q2N4P6R3T5W8Y0Z1
note_id	TEXT NOT NULL	FK to BIL-02's credit_note.note_id or debit_note.note_id (issued by GEN-01). Example: cn_01HZX5M2K9P3Q7R4T6W8Y0Z2N4 for a credit note, dn_01HZX5M2K9P3Q7R4T6W8Y0Z2N4 for a debit note
note_type	TEXT NOT NULL	Enum. Values: CREDIT (reduces outstanding) / DEBIT (increases outstanding)
adjustment_request_id	TEXT NOT NULL	FK to ADJ-01's adjustment_request.id (the proposal that led to this note). Example: adj_01HZX5M2K9P3Q7R4T6W8Y0Z2N4
customer_id	TEXT NOT NULL	Example: cust_01HX3K9M2P5Q8R1T4W7Y0Z3N6
operator_code	TEXT NOT NULL	Values: WIK / WUG / WTZ / YASSN
note_amount	NUMERIC(18,2) NOT NULL	Face value of the note as issued by GEN-01 (always positive). Example: 1500.00
currency	TEXT NOT NULL	ISO 4217. Example: KES
target_kind	TEXT NOT NULL	Enum. Values: INVOICE (applied to a POSTPAID invoice's outstanding) / WALLET (credited/debited a PREPAID wallet via BIL-05) / CREDIT_BALANCE (credit note surplus → account_credit_balance owned by PAY-01)
target_id	TEXT NULL	The target identifier. For INVOICE target: invoice_id (e.g. inv_01HZ. . .). For WALLET target: wallet_ref from BIL-05 (e.g. wal_01HZ. . .). For CREDIT_BALANCE target: NULL (the credit balance is keyed by customer_id, no separate ID)
applied_amount	NUMERIC(18,2) NOT NULL	What was actually applied to this target. For an invoice target, capped at min(note_amount_remaining, invoice_outstanding). Example: 1500.00
target_balance_before	NUMERIC(18,2) NOT NULL	Pre-application balance — invoice outstanding, wallet balance, or credit balance. Example: 2500.00 (an invoice's outstanding before credit note applied)
target_balance_after	NUMERIC(18,2) NOT NULL	Post-application balance. Example: 1000.00 (outstanding reduced by 1500 credit)
status	TEXT NOT NULL	Enum. Values: APPLIED (successful) / FAILED (e.g., PREPAID debit on insufficient wallet, missing invoice)
failure_reason	TEXT NULL	Enum if status = FAILED. Values: WALLET_INSUFFICIENT_BALANCE (PREPAID debit, wallet less than note amount) / INVOICE_NOT_FOUND (target_invoice_id refers to non-existent invoice) / INVOICE_NOT_APPLIABLE (invoice is CANCELLED or WRITTEN_OFF) / CURRENCY_MISMATCH
applied_at	TIMESTAMP NOT NULL DEFAULT NOW()	Example: 2026-05-19 15:00:00

Indexes: PK; UNIQUE on (note_id, target_kind, target_id) to support split applications (one note → multiple rows when applying to invoice + surplus to credit balance, or auto-allocated across multiple invoices); (adjustment_request_id); (customer_id, applied_at DESC); (target_kind, target_id, applied_at DESC) WHERE target_id IS NOT NULL. P10Y retention.

Sample data:

id	note_id	note_type	target_kind	target_id	applied_amount	balance_before	balance_after	status
na_01HZ...1	cn_01HZ...A	CREDIT	INVOICE	inv_01HZ...X	1500.00	2500.00	1000.00	APPLIED
na_01HZ...2	cn_01HZ...B	CREDIT	WALLET	wal_01HZ...M	800.00	200.00	1000.00	APPLIED
na_01HZ...3	dn_01HZ...C	DEBIT	INVOICE	inv_01HZ...Y	500.00	0.00	500.00	APPLIED
na_01HZ...4	dn_01HZ...D	DEBIT	WALLET	wal_01HZ...N	1000.00	300.00	300.00	FAILED
na_01HZ...5	cn_01HZ...E	CREDIT	INVOICE	inv_01HZ...P	2000.00	2000.00	0.00	APPLIED
na_01HZ...6	cn_01HZ...E	CREDIT	CREDIT_BALANCE	NULL	500.00	0.00	500.00	APPLIED

How to read: Row 1 — 1500 KES credit note applied to a POSTPAID invoice, reducing outstanding from 2500 to 1000. Row 2 — 800 KES credit note applied to a PREPAID wallet. Row 3 — 500 KES debit note added to a previously-PAID POSTPAID invoice (outstanding goes from 0 to 500; STA-01 transitions PAID → PENDING/OVERDUE based on due date). Row 4 — 1000 KES debit note against a PREPAID wallet failed because the wallet had only 300 (couldn't go negative). Rows 5 and 6 — a single 2500 KES credit note (cn_01HZ...E) split across TWO targets: 2000 went to clear an invoice fully, and the surplus 500 went to account_credit_balance (which is owned by PAY-01).

Event payloads

sophix.billing.creditnote.applied

POSTPAID invoice target:

```
{
  "eventType": "CreditNoteApplied",
  "aggregateId": "cn_01HZ...A",
  "timestamp": "2026-05-19T15:00:00Z",
  "payload": {
    "noteId": "cn_01HZ...A",
    "adjustmentRequestId": "adj_01HZ...",
    "customerId": "cust_01HX...",
    "operatorCode": "WIK",
    "target": {
      "type": "INVOICE",
      "invoiceId": "inv_01HZ...X",
      "appliedAmount": 1500.00,
      "outstandingAfter": 1000.00
    },
    "currency": "KES",
    "appliedAt": "2026-05-19T15:00:00Z"
  }
}
```

Credit balance surplus target:

```
{
  "eventType": "CreditNoteApplied",
  "aggregateId": "cn_01HZ...E",
  "timestamp": "2026-05-19T15:30:02Z",
  "payload": {
    "noteId": "cn_01HZ...E",
    "adjustmentRequestId": "adj_01HZ...",
    "customerId": "cust_01HY...",
    "operatorCode": "WIK",
    "target": {
      "type": "CREDIT_BALANCE",
      "appliedAmount": 500.00,
      "balanceAfter": 500.00
    },
    "currency": "KES",
    "appliedAt": "2026-05-19T15:30:02Z"
  }
}
```

sophix.billing.debitnote.applied

```
{
  "eventType": "DebitNoteApplied",
  "aggregateId": "dn_01HZ...C",
  "timestamp": "2026-05-19T16:00:00Z",
  "payload": {
```

```

      "noteId": "dn_01HZ...C",
      "adjustmentRequestId": "adj_01HZ...",
      "customerId": "cust_01HY...",
      "operatorCode": "WIK",
      "target": {
        "type": "INVOICE",
        "invoiceId": "inv_01HZ...Y",
        "appliedAmount": 500.00,
        "outstandingAfter": 500.00
      },
      "currency": "KES",
      "appliedAt": "2026-05-19T16:00:00Z"
    }
  }
}

```

sophix.billing.debitnote.application-failed

```

{
  "eventType": "DebitNoteApplicationFailed",
  "aggregateId": "dn_01HZ...D",
  "timestamp": "2026-05-19T16:30:00Z",
  "payload": {
    "noteId": "dn_01HZ...D",
    "adjustmentRequestId": "adj_01HZ...",
    "customerId": "cust_01HZ...",
    "operatorCode": "WIK",
    "target": {
      "type": "WALLET",
      "walletRef": "wal_01HZ...N"
    },
    "attemptedAmount": 1000.00,
    "walletBalance": 300.00,
    "currency": "KES",
    "reason": "WALLET_INSUFFICIENT_BALANCE",
    "attemptedAt": "2026-05-19T16:30:00Z"
  }
}

```

Cross-DD coordination

Touched	What CN-01 owns	What the other DD owns
BIL-01 Core (parent)	Shares the billing-charging module deployment.	Owns charging engine.
BIL-01-PAY-01 (sibling)	Calls into PAY-01's CreditBalanceService for credit note surplus.	Owns account_credit_balance table.
BIL-02-Core	Reads invoice.outstanding_amount for applicability check and updates outstanding_amount for POSTPAID note application.	Owns the invoice aggregate.
BIL-02-GEN-01	Consumes CreditNoteIssued / DebitNoteIssued events.	Owns note issuance.
BIL-02-ADJ-01	No direct interaction. ADJ-01 originates the adjustment, but the actual note documents come through GEN-01.	Owns adjustment workflow.
BIL-02-STA-01	Emits CreditNoteApplied / DebitNoteApplied / DebitNoteApplicationFailed. STA-01 consumes for invoice status transitions.	Owns invoice status.
BIL-05 Wallet and Topup	Calls POST /api/wallets/credit for PREPAID credit notes, POST /api/wallets/debit for PREPAID debit notes.	Owns wallet ops.
DD_NOT-01 Notification Service	Emits note events NOT-01 may consume for customer notifications.	Owns notification.

Dependencies

Dependency	Owner	Status
BIL-01 Charging Engine (Core)	billing team	✓ v0.9 (sibling)
BIL-01-PAY-01 (sibling — for CreditBalanceService)	billing team	✓ v1.0 (sibling)
BIL-02-Core	billing team	✓ v2.11
BIL-02-GEN-01	billing team	✓ v1.3

Dependency	Owner	Status
BIL-02-ADJ-01	billing team	✓ v1.3
BIL-02-STA-01	billing team	✓ v1.0
BIL-05 Wallet and Topup	billing team	✓ v1.0
DD_NOT-01 Notification Service	notification team	✓ v1.3
Foundation: Kafka, DB, Auth	infra team	✓

B — Current TZ

Workshop materials do not describe a structured credit / debit note application service in TZ Sophix. Adjustments are likely handled by direct DB updates against invoice tables or wallet logs with no centralized ledger or event emission.

C — Gap & bridge

CN-01 is new functionality. It introduces:

- Event-driven application of credit / debit notes from ADJ-01 / GEN-01
- Configurable surplus handling (reuses PAY-01's overpayment policy for credit note surplus)
- Per-target audit in `note_application_ledger` with split rows for surplus
- Formalized event contracts (`CreditNoteApplied`, `DebitNoteApplied`, `DebitNoteApplicationFailed`, `NoteApplicationFailed`) consumed by STA-01, BIL-04, NOT-01, analytics
- Safe handling of PREPAID debit notes against insufficient wallets (no negative balances)